Experimental Evaluation of Chunked Fixed-Priority and EDF Scheduling on STM32

Vladimir Toporkov, Artem Burmyakov
Innopolis University

I. Introduction

1.Intro: Real time systems, STM32, What this paper examines? example: AB 2015 article

STM32 is a family of microcontrollers developed by STMicroelectronics for embedded applications. These devices integrate a processor, memory, and peripheral interfaces on a single chip [1].

STM32-based devices support applications such as environmental monitoring [2] and equipment condition monitoring [3]. In these applications maintaining responsiveness as the workload increases is a key challenge.

The paper examines how the choice of scheduling approach affects responsiveness and processing cost on STM32. We compare two approaches: Superloop [4] and Earliest Deadline First (EDF) [5].

A. Related works

Previous studies have examined cyclic and cooperative scheduling approaches for embedded systems. Koopman [6] describes pure static scheduling, where tasks execute sequentially in a fixed order within a main loop. This approach is simple to implement and provides predictable execution. However, long-running tasks can delay subsequent tasks and cause deadline misses. Locke [7] compares cyclic executives with fixed-priority executives. His analysis highlights that modifying predefined cyclic schedules can be difficult when task requirements change. Unlike predefined cyclic schedules, the Superloop considered in this work checks the readiness of periodic tasks in a fixed order and executes them when they are ready.

The limitations of cooperative scheduling have also been examined in practical applications. Jung et al. [8] implemented a cooperative main loop for UAV control, where computationally intensive path-planning tasks were divided into smaller execution slices to meet the timing requirements of periodic control tasks. However, manually managing task execution times was complex, which motivated the use of a preemptive real-time kernel. Short [9] analyzed TTC and TTH schedulers and identified limitations in their scheduling overhead and schedulability. The proposed modifications reduced scheduling overhead and increased the proportion of schedulable task sets. Nevertheless, non-preemptive EDF achieved better schedulability in the evaluated scenarios.

Liu and Layland [5] analyzed Earliest Deadline First (EDF) scheduling, which selects the ready job with the earliest absolute deadline. They proved that, for independent periodic tasks with deadlines equal to their periods, preemptive EDF can schedule a task set on a single processor if and only if its total utilization does not exceed 100%. However, this theoretical result assumes unrestricted preemption and does not explicitly model preemption overhead.

To reduce the costs of preemption, Bertogna and Baruah [10] proposed a limited-preemption EDF scheduler that delays preemptions when possible. Their simulations showed that, for task sets with ten tasks, the average number of preemptions was less than 20% of that under fully preemptive EDF. Other studies have considered limiting preemption to specific points during task execution. Buttazzo et al. [11] reviewed several limited-preemption approaches, including fixed preemption points, where tasks are divided into non-preemptive chunks. Bertogna et al. [12] proposed a method for placing these points to balance blocking time and preemption overhead. These studies demonstrate the importance of controlling when preemptions can occur, although their evaluations primarily rely on analytical models and simulations. The Chunked EDF considered in this work follows a related principle by selecting the earliest-deadline job after each non-preemptive chunk.

Experimental studies have also evaluated scheduling approaches on resource-constrained embedded platforms. Baynes et al. [13] compared a minimal multirate scheduler with cooperative and preemptive RTOSs using a processor simulator. Their results showed no clear advantage of either cooperative or preemptive scheduling in timing accuracy, while the minimal scheduler had lower energy overhead. Marau et al. [14] implemented a minimal real-time kernel on a PIC18 microcontroller and showed that EDF scheduling overhead increased with the number of simultaneously released tasks. Oliveira and Lima [15] evaluated two EDF implementations on an STM32F407 running FreeRTOS. Their experiments showed that the implementation of the ready queue significantly affected scheduling overhead and deadline misses.

2.Garbage gap?

However, the reviewed experiments do not directly compare Superloop and Chunked EDF within the same bare-metal implementation. This work addresses this gap by evaluating both approaches on an STM32F767 under different processor utilization levels and, for Chunked EDF, different maximum chunk durations Δ, measuring response times, deadline misses and skipped releases.

B. Contribution

First, we compare Superloop and chunked EDF for periodic tasks with implicit deadlines on an STM32F767 microcontroller. Then, we evaluate the effects of task-set utilization on response time, deadline misses, skipped releases, and measured task-selection overhead. Also, we investigate how maximum chunk duration affects these metrics for chunked EDF. Both scheduler implementations are publicly available.

C. Paper organization

Initially, in Section II, we provide the necessary background on STM32. We describe the hardware platform in Section II-A and the project structure in Section II-B. Sections II-C and **??** describe how application functions execute and explain the differences between synthetic and sensor workloads. Next, in Section III we formalize the periodic task model used in our experiments. Finally, in Section IV, we describe Superloop, fully preemptive EDF, and chunked EDF and illustrate their schedules for the same task set.

II. Background on STM32

Let us first provide the necessary preliminaries on the hardware and software components, which we later use in our evaluation for schedulers.

A. Hardware platform

The central component of our hardware platform is the STM32F767 microcontroller, shown in Fig. 1. It has the following characteristics:

- It is a single-core microcontroller.
- It uses an ARM Cortex-M7 processor core.
- Its maximum clock frequency is 216 MHz.

To generate workloads for the STM32 microcontroller, we use several sensors, summarized in Table I. Each sensor has the following properties:

- It is connected to the STM32 through one of the hardware interfaces listed in Table I.
- It is associated with a periodic task that generates a sequence of jobs, as formally described in Section III.
- Its task shares the same processor with other sensor tasks, requiring a scheduler to manage their execution.

The code responsible for reading and processing sensor data forms the sensor-based workload used in our scheduling experiments.

TABLE I
Sensor functions and interfaces.

Sensor	Purpose	Interface
DHT11 [16]	Temperature and humidity	Single-wire digital, GPIO
HC-SR04 [17]	Distance	Trigger/echo, GPIO
OV2640 [18]	Image acquisition	Parallel data, SCCB
SPW2430 [19]	Signal acquisition	Analog output, ADC

GPIO - General-Purpose Input/Output; SCCB - Serial Camera Control Bus; ADC - Analog-to-Digital Converter.

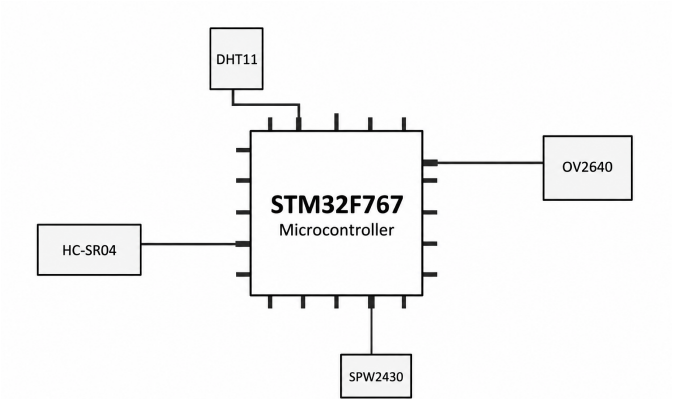


Fig. 1. Sensor platform based on STM32F767.

B. STM32 Software

To work with STM32 hardware, we use specialized software tools developed by STMicroelectronics. These tools simplify the development process for STM32 microcontrollers.

The first tool is STM32CubeMX. It allows us to:

- Configure the microcontroller clock.
- Configure pins and peripherals.
- Generate a C project with initialization code.

Fig. 2 shows an example of the generated project structure. One of the main files is `main.c`, which contains initialization code and special sections marked as `USER CODE`. These sections allow us to preserve our code during project regeneration. Changes made outside these sections may be overwritten.

After configuring and generating the project, we use STM32CubeIDE to:

- Write application code.
- Build the project.
- Debug the program on the microcontroller.

We will use this tools to implement algorithms described in Section IV

3.Shrink the tree?

```
example_project/
├── Core/ ..... Application code
│   ├── Inc/ ..... Header files
│   └── Src/ ..... Source files
│       ├── main.c ..... Init. and main loop
│       ├── stm32f7xx_it.c ..... Interrupt handlers
│       ├── stm32f7xx_hal_msp.c ..... Low-level peripheral setup
│       └── system_stm32f7xx.c ..... Core init. and clock state
│   └── Startup/ ..... Startup code and interrupt vectors
├── Drivers/ ..... Hardware support
│   └── CMSIS/ ..... Core and MCU definitions
├── STM32F767ZITX_FLASH.ld
├── STM32F767ZITX_RAM.ld
└── test_project.ioc ..... CubeMX configuration
```

Fig. 2. Main components of the STM32CubeMX-generated project.

C. Default execution model

After configuring the STM32F767 project in STM32CubeMX, a default Superloop is generated in `main.c`. It is an infinite `while(1)` loop and serves as the main execution loop of the project. The Superloop has the following characteristics:

- Fixed execution order: functions are called sequentially in a predefined order.
- Non-preemptive execution: each function runs to completion before the next one starts.
- Repeated execution: once all functions have finished, the loop starts a new iteration.

Fig. 3 illustrates Superloop execution scenario. The example includes two functions: `execute_taskA()` and `execute_taskB()`. Both functions generate synthetic CPU load. Their execution times are 6 ms and 4 ms, respectively. The first function executes before the second one. Therefore, `execute_taskB()` must wait until `execute_taskA()` finishes. Each cycle takes 10 ms to be completed. The sequence repeats until the program is stopped.

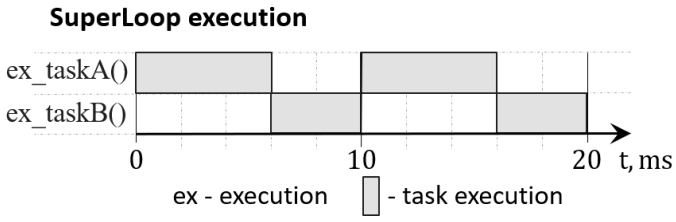


Fig. 3. Example of Superloop execution with two sequential functions.

Fig. 3 shows the basic execution principle of Superloop. However, the sensors listed in Table I have specific timing requirements. According to their documentation, sensor operations must be performed at certain time intervals. We discuss these requirements in the Subsection II-D.

D. Synthetic and sensor workloads

III. System Model

We use the periodic task model of Liu and Layland [5]. We also adapt the task and release notation of Burmyakov and Bini [20]. Table II summarizes the notation.

A real-time system $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ consists of n periodic tasks executed on one core ($m = 1$). Each task $\tau_i = (C_i, D_i, P_i)$ is defined by the worst-case execution time C_i of a job, a relative deadline D_i, and a period P_i. We consider implicit deadlines, $D_i = P_i$.

Let $J_{i,j}$ denote the j-th job of τ_i, released at $r_{i,j}$. Consecutive jobs are released one period apart:

$$r_{i,j+1} = r_{i,j} + P_i. \tag{1}$$

Each job $J_{i,j}$ has an absolute deadline $d_{i,j} = r_{i,j} + D_i$. If completed, its finish time is denoted by $f_{i,j}$.

TABLE II
System model notation.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Worst-case execution time of a job of τ_i
P_i	Task period
$D_i = P_i$	Relative deadline, equal to the period
$J_{i,j}$	The j-th job of task τ_i
$r_{i,j}$	Release time of job $J_{i,j}$
$f_{i,j}$	Job completion time
$d_{i,j} = r_{i,j} + D_i$	Absolute deadline of job $J_{i,j}$
m	Number of cores
$U_i = C_i/P_i$	Task utilization
$U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$	Task-set utilization

- A schedule specifies which job executes at each time, if any.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job $J_{i,j}$ misses its deadline if $f_{i,j} > d_{i,j}$.
- The response time of a completed job $J_{i,j}$ is $f_{i,j} - r_{i,j}$.

IV. Superloop and EDF Schedulers

We next describe the Superloop and chunked EDF schedulers used in our experiments. Fully preemptive EDF serves as a reference for comparison.

A. Superloop

Our Superloop implementation supports periodic task execution. It follows three rules:

- Periodic tasks: each task can run only after its next release time.
- Fixed order: ready tasks run in the same order.
- Non-preemptive: each job runs until it finishes.

At the start of each loop iteration, the scheduler checks all tasks using the same current time. A task that becomes ready during execution is considered in the next iteration.

Similar approaches to periodic task scheduling in a main loop are described by Koopman [21] and Jones [22]. Algorithm 1 shows this process.

Algorithm 1 Superloop Scheduler

```
function SUPERLOOP(𝒯)
    t_i^next ← 0,   i = 1, ..., n                ▷ Initial releases
    while true do                                 ▷ Infinite cycle
        t^cur ← NOW()                             ▷ Get current system time
        for i ← 1 to n do
            ready_i ← (t_i^next ≤ t^cur)          ▷ Save readiness
        end for
        for i ← 1 to n do
            if ready_i then
                J_{i,j} ← (τ_i, t_i^next)         ▷ Keep release time
                EXECUTE(J_{i,j})
                f_{i,j} ← NOW()
                t_i^next ← t_i^next + P_i         ▷ Advance, then skip
                while t_i^next < f_{i,j} do t_i^next ← t_i^next + P_i
            end if
        end for
    end while
end function
```

Fig. 4(a) illustrates the Superloop schedule for tasks τ_1 and τ_2. Both tasks release their first jobs at $t = 0$. Since τ_1 comes first in the polling order, $J_{1,1}$ executes until $t = 18$ ms. Job $J_{2,1}$ then starts and completes at $f_{2,1} = 28$ ms, missing its deadline $d_{2,1} = 20$ ms.

The release at $r_{2,2} = 20$ ms occurs while $J_{2,1}$ is still running. Once this job completes, the scheduler counts that release as skipped and advances the next release time to $r_{2,3} = 40$ ms. Polling continues with no ready job until then.

B. Earliest Deadline First

In contrast, fully preemptive EDF selects the ready job with the earliest absolute deadline. A newly released job preempts the running job if its deadline is earlier.

Fig. 4(b) shows the EDF schedule for the same task set. At $t = 0$, $J_{2,1}$ has the earlier deadline and executes until $t = 10$ ms. Job $J_{1,1}$ then starts, but is preempted at $r_{2,2} = 20$ ms because the new job has a deadline $d_{2,2} = 40$ ms, earlier than $d_{1,1} = 50$ ms. Job $J_{2,2}$ executes until $t = 30$ ms, after which $J_{1,1}$ resumes and completes at $t = 38$ ms. The next job of τ_2 executes from $t = 40$ to $t = 50$ ms. Thus, no deadline is missed and no release is skipped in this example.

C. Chunked Earliest Deadline First

Unlike fully preemptive EDF, our chunked EDF implementation allows switching between jobs only after an execution step finishes. Its key features are:

- Earliest deadline first: the ready job with the earliest absolute deadline is selected.
- Chunked execution: jobs run in non-preemptive steps. For computational workloads, each chunk lasts at most Δ ms.
- Repeated selection: the scheduler selects a job after each execution step.

Algorithm 2 shows this process. The Boolean flag a_i indicates whether task τ_i has an unfinished job. At each iteration, $\mathcal{R}$ contains the indices of runnable tasks. These include tasks with due releases and active jobs that can resume. Jobs waiting for an external event are excluded.

For an active job, $d_i = r_i + D_i$, where r_i is its saved release time. Otherwise, $d_i = t_i^{\text{next}} + D_i$.

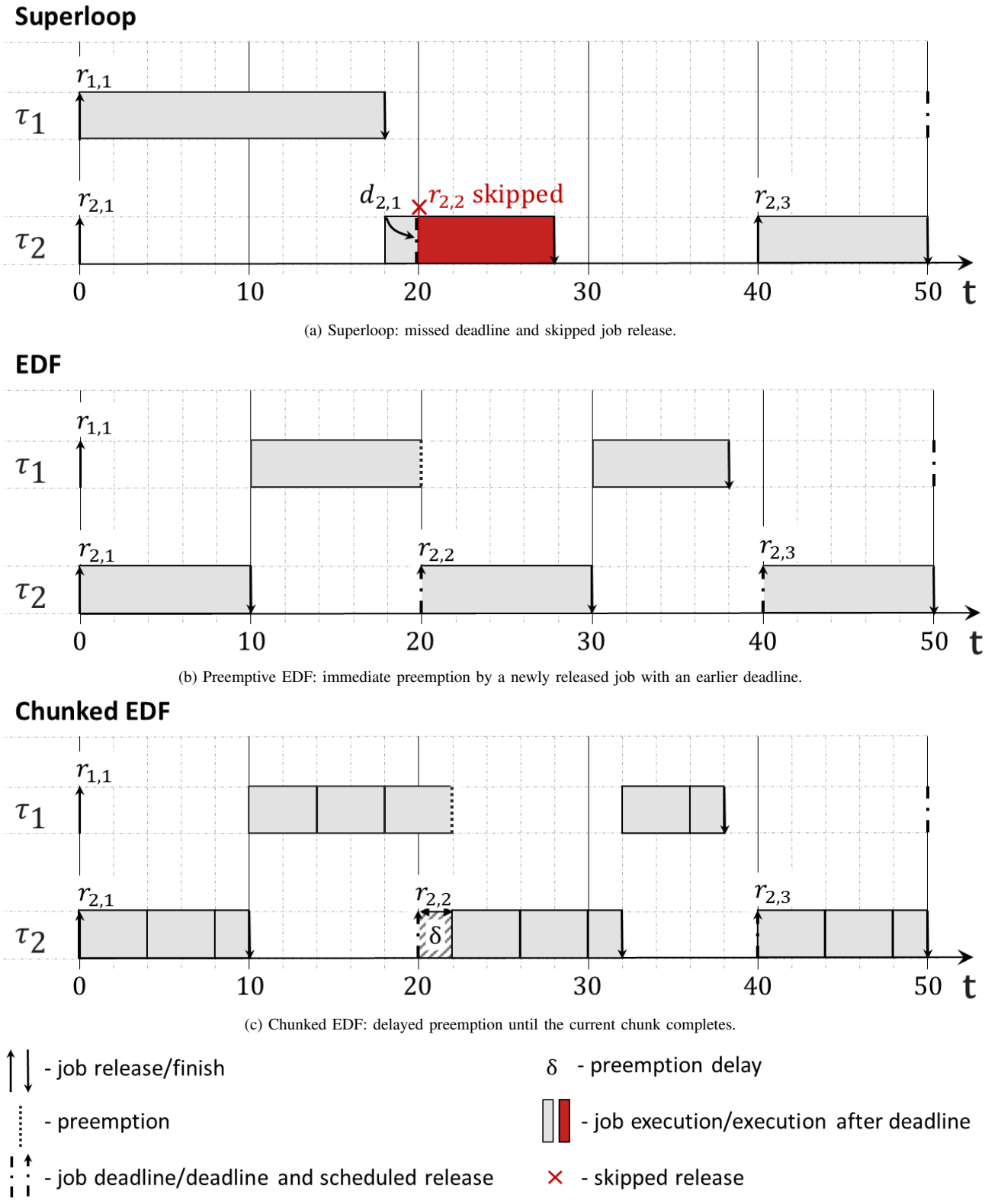
Algorithm 2 Chunked EDF Scheduler

```
function CHUNKEDEDF(𝒯, Δ)
    t_i^next ← 0,   hasActiveJob_i ← false,   i = 1, ..., n
    while true do                                 ▷ Infinite cycle
        t^cur ← NOW()                             ▷ Get current system time
        ℛ ← {i | READY(i, t^cur)}                 ▷ Ready task indices
        if ℛ ≠ ∅ then
            i* ← arg min_{i∈ℛ} d_i                ▷ Earliest deadline
            if ¬hasActiveJob_{i*} then
                r_{i*} ← t_{i*}^next;
                hasActiveJob_{i*} ← true          ▷ Save release
            end if
            EXECUTESTEP(i*, Δ)                    ▷ At most Δ
            if COMPLETED(i*) then
                f ← NOW()
                t_{i*}^next ← t_{i*}^next + P_{i*}   ▷ Advance, then skip
                while t_{i*}^next < f do t_{i*}^next ← t_{i*}^next + P_{i*}
                hasActiveJob_{i*} ← false
            end if
        end if
    end while
end function
```

Fig. 4(c) shows the same task set with $\Delta = 4$ ms. At $r_{2,2} = 20$ ms, job $J_{1,1}$ is still executing a chunk. This chunk ends at $t = 22$ ms. Job $J_{2,2}$ then starts with a preemption delay of $\delta = 2$ ms. It executes in chunks of 4, 4, and 2 ms and completes at $t = 32$ ms. The shorter final chunk reflects the remaining work. Job $J_{1,1}$ then resumes and completes at $t = 38$ ms.

$$\tau_i = (C_i, D_i, P_i), \qquad D_i = P_i, \qquad m = 1$$
$$\tau_1 = (18, 50, 50), \qquad \tau_2 = (10, 20, 20)$$

All times in ms. Maximum chunk duration in (c): $\Delta = 4$ ms.



(a) Superloop: missed deadline and skipped job release.

(b) Preemptive EDF: immediate preemption by a newly released job with an earlier deadline.

(c) Chunked EDF: delayed preemption until the current chunk completes.

Fig. 4. Comparison of Superloop, preemptive EDF, and chunked EDF schedules for the same periodic task set.

V. Performance evaluation

A. Synthetic workload

B. Real sensors workload

References

[1] *STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: Datasheet*, STMicroelectronics. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f767zi.pdf

[2] STMicroelectronics, "Process and environment monitoring," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/process-and-environment-monitoring.html

[3] ——, "Condition monitoring / predictive maintenance," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/condition-monitoring-predictive-maintenance.html

[4] Arm Keil, "Execution models: Single task system (Superloop)," accessed: Oct. 4, 2026. [Online]. Available: https://www.keil.com/pack/doc/mw/Graphic/html/gui_exec_model.html

[5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[6] P. Koopman, *Better Embedded System Software*, 1st ed. Independently published, 2021, revised 2021, version 1.1. [Online]. Available: https://betterembsw.blogspot.com/2021/02/better-embedded-system-software-e-book.html

[7] C. D. Locke, "Software architecture for hard real-time applications: Cyclic executives vs. fixed priority executives," *Real-Time Systems*, vol. 4, no. 1, pp. 37–53, Mar. 1992.

[8] D. Jung, J. Ratti, and P. Tsiotras, "Real-time implementation and validation of a new hierarchical path planning scheme of UAVs via hardware-in-the-loop simulation," *Journal of Intelligent & Robotic Systems*, vol. 54, no. 1–3, pp. 163–181, Mar. 2009.

[9] M. Short, "Analysis and redesign of the 'TTC' and 'TTH' schedulers," *Journal of Systems Architecture*, vol. 58, no. 1, pp. 38–47, Jan. 2012.

[10] M. Bertogna and S. Baruah, "Limited preemption EDF scheduling of sporadic task systems," *IEEE Transactions on Industrial Informatics*, vol. 6, no. 4, pp. 579–591, Nov. 2010.

[11] G. C. Buttazzo, M. Bertogna, and G. Yao, "Limited preemptive scheduling for real-time systems. A survey," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 1, pp. 3–15, Feb. 2013.

[12] M. Bertogna, G. Buttazzo, M. Marinoni, G. Yao, F. Esposito, and M. Caccamo, "Preemption points placement for sporadic task sets," in *Proceedings of the 22nd Euromicro Conference on Real-Time Systems (ECRTS)*. Brussels, Belgium: IEEE, Jul. 2010, pp. 251–260.

[13] K. Baynes, C. Collins, E. Fiterman, B. Ganesh, P. Kohout, C. Smit, T. Zhang, and B. Jacob, "The performance and energy consumption of embedded real-time operating systems," *IEEE Transactions on Computers*, vol. 52, no. 11, pp. 1454–1469, Nov. 2003.

[14] R. Marau, P. Leite, M. Velasco, P. Martí, L. Almeida, P. Pedreiras, and J. M. Fuertes, "Performing flexible control on low-cost microcontrollers using a minimal real-time kernel," *IEEE Transactions on Industrial Informatics*, vol. 4, no. 2, pp. 125–133, May 2008.

[15] G. Oliveira and G. Lima, "Scheduling and energy savings for small scale embedded FreeRTOS-based real-time systems," *Design Automation for Embedded Systems*, vol. 27, no. 1–2, pp. 3–29, Jun. 2023.

[16] *DHT11 Humidity & Temperature Sensor: Technical Data Sheet*, Aosong Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[17] *Ultrasonic Ranging Module HC-SR04*, Elecfreaks. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[18] *OV2640 Datasheet*, OmniVision Technologies. [Online]. Available: https://www.uctronics.com/download/OV2640_DS.pdf

[19] *SPW2430HR5H-B: Top Port SiSonic Microphone*, Knowles Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/218/SPW2430HR5H-B-1290924.pdf

[20] A. Burmyakov and E. Bini, "Improving the efficiency of global EDF exact schedulability test," *IEEE Transactions on Computers*, pp. 1–14, 2026.

[21] P. Koopman, "Multi-rate main loop tasking," Dec. 2015, accessed: Oct. 19, 2026. [Online]. Available: https://betterembsw.blogspot.com/2015/12/multi-rate-main-loop-tasking.html

[22] N. Jones. (2024) You don't need an RTOS (part 1). Accessed: Oct. 19, 2026. [Online]. Available: https://www.embeddedrelated.com/showarticle/1636.php